

Session 13 : Les moteurs physiques commerciaux

Objectifs de la session

- Comprendre pourquoi on utilise des moteurs physiques existants en production.
- Découvrir les moteurs majeurs et leurs cas d'usage.
- Introduire le déterminisme et l'architecture ECS.

1. Pourquoi utiliser un moteur physique ?

Avantages des moteurs existants

- **Complexité** : gestion des contacts multiples, stabilité, optimisation.
- **Temps** : des décennies de R&D.
- **Performance** : optimisé pour des milliers d'objets.

2. Concepts avancés

Rigid Body vs Soft Body

- **Rigid Body** : objet indéformable (boîte, sphère).
- **Soft Body** : objet déformable (tissu, gelée), beaucoup plus complexe.

Continuous Collision Detection (CCD)

Problème : un objet rapide peut traverser un mur entre deux frames (tunneling). **Solution** : on raycaste la trajectoire entre t et $t + \Delta t$ au lieu de tester seulement les positions.

Sleeping

Si un objet est immobile, on arrête de le simuler. Il "dort" et se réveille si un autre objet le touche.

3. Comparatif des moteurs

Moteur	Langage	Usage
PhysX	C++	AAA Games (Unity, Unreal default)
Havok	C++	AAA Games (Half-Life 2, Zelda TotK)
Jolt Physics	C++	Modern, open-source, Horizon Forbidden West
Box2D	C++	2D Games (Angry Birds, Terraria)
Cannon.js	JavaScript	Three.js (simple, legacy)
Ammo.js	JavaScript (WASM)	Three.js (Bullet port)
Rapier	Rust/WASM	Modern, fast, Three.js recommended

Table 1: Comparatif des moteurs physiques

4. Déterminisme

Déterminisme

Un moteur est déterministe si les mêmes inputs produisent toujours les mêmes outputs.

Importance : multijoueur, replay, replays de gameplay.

Pièges : ordre des opérations, précision flottante selon le CPU.

5. Architecture ECS

Entity-Component-System

- **Entity** : un simple ID (ex: entity_42).
- **Component** : données pures (ex: Position, Rigidbody, Mesh).
- **System** : logique qui opère sur les composants (ex: PhysicsSystem, RenderSystem).

Avantage : flexibilité, on ajoute/enlève des composants à la volée.